PropertyTrust

Property Insurance Management System

Gurumetkal Bharath Simha Reddy

Guidance: Ranjan Bhatnagar

Abstract

PropertyTrust is an end-to-end Property Insurance Management System designed to modernize the insurance lifecycle through automation and artificial intelligence. The project provides a feature-rich digital ecosystem where Agents can seamlessly onboard customers and manage property policies, while Customers can digitally submit claims with real-time status tracking. A central innovation of the system is its hybrid AI-driven fraud detection engine, which utilizes a Python microservice integrated with the OpenRouter LLM to automatically analyze and score insurance claims for risk. Built with .NET 8 Clean Architecture and a reactive Angular frontend, the project ensures secure, role-based access for all stakeholders—including Admins and Claims Officers—while maintaining a robust audit trail of every transaction to ensure transparency and regulatory compliance.

Technology Stack

Our robust architecture leverages modern, scalable technologies to ensure secure and efficient insurance operations.

Angular 20+

Modern frontend framework with component-based architecture for responsive user experience.

.NET Web API

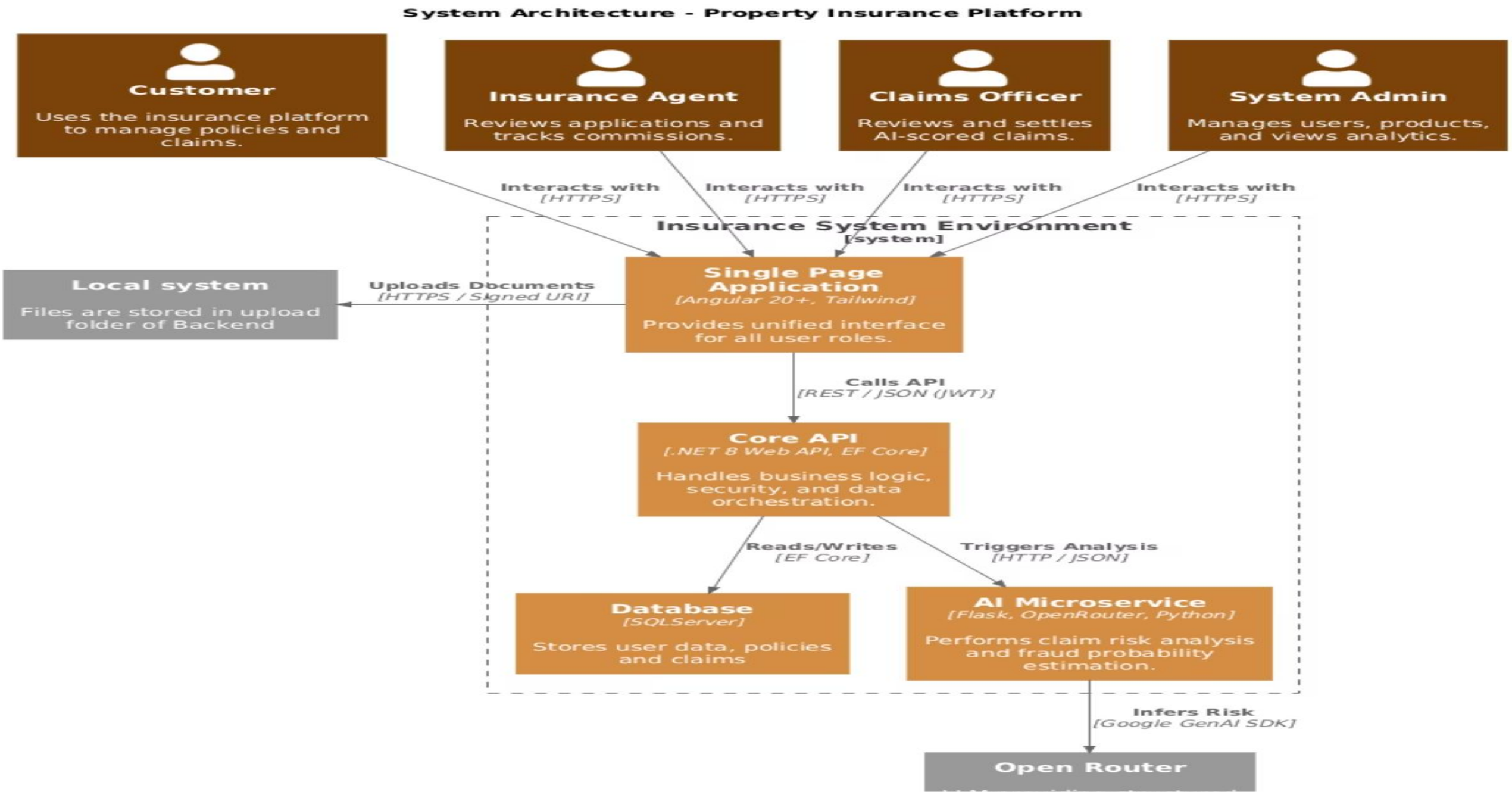
Backend services providing RESTful endpoints with enterprise-grade security and scalability.

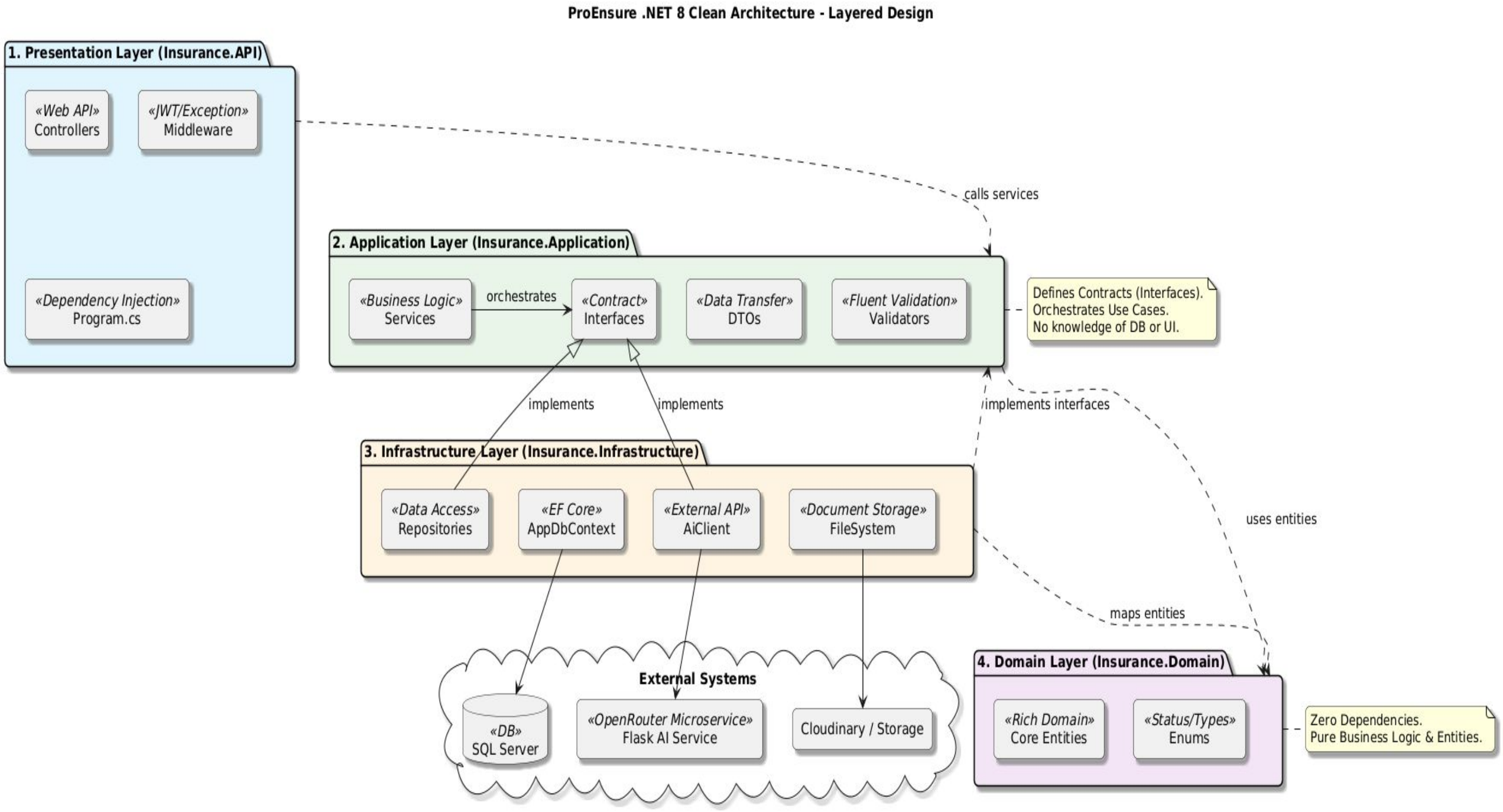
Flask AI Microservice

Machine learning models for risk scoring, fraud detection, and decision support.

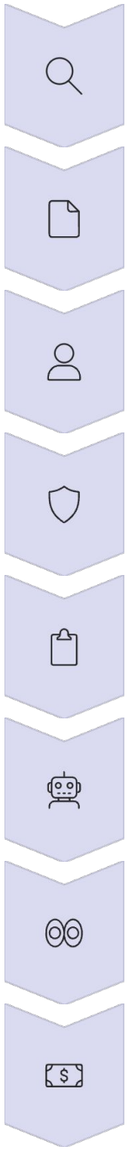
SQL Server

Relational database for secure data storage with ACID compliance and robust relationships.


System Architecture - Property Insurance Platform
Customer
Uses the insurance platform to manage policies and claims.
Insurance Agent
Reviews applications and tracks commissions.
Claims Officer
Reviews and settles AI-scored claims.
System Admin
Manages users, products, and views analytics.
Interacts with
[HTTPS]
Interacts with
[HTTPS]
Interacts with
[HTTPS]
Interacts with
[HTTPS]
Insurance System Environment
[system]
Local system
Files are stored in upload folder of Backend
Uploads Documents
[HTTPS / Signed URI]
Single Page Application
[Angular 20+, Tailwind]
Provides unified interface for all user roles.
Calls API
[REST / JSON (JWT)]
Core API
[.NET 8 Web API, EF Core]
Handles business logic, security, and data orchestration.
Reads/Writes
[EF Core]
Triggers Analysis
[HTTP / JSON]
Database
[SQLServer]
Stores user data, policies and claims
AI Microservice
[Flask, OpenRouter, Python]
Performs claim risk analysis and fraud probability estimation.
Infers Risk
[Google GenAI SDK]
Open Router


ProEnsure .NET 8 Clean Architecture - Layered Design
1. Presentation Layer (Insurance.API)
«Web API»
Controllers
«JWT/Exception»
Middleware
«Dependency Injection»
Program.cs
calls services
2. Application Layer (Insurance.Application)
«Business Logic»
Services
orchestrates
«Contract»
Interfaces
«Data Transfer»
DTOs
«Fluent Validation»
Validators
Defines Contracts (Interfaces).
Orchestrates Use Cases.
No knowledge of DB or UI.
implements
implements
implements interfaces
3. Infrastructure Layer (Insurance.Infrastructure)
«Data Access»
Repositories
«EF Core»
AppDbContext
«External API»
AiClient
«Document Storage»
FileSystem
uses entities
maps entities
External Systems
«DB»
SQL Server
«OpenRouter Microservice»
Flask AI Service
Cloudinary / Storage
4. Domain Layer (Insurance.Domain)
«Rich Domain»
Core Entities
«Status/Types»
Enums
Zero Dependencies.
Pure Business Logic & Entities.

Business Workflow



Product Discovery

Application Submission

Agent Approval

Policy Activation

Claim Submission

AI Risk Scoring

Claims Review

Settlement & Commission

Role-Based Responsibilities

Customer

- Browse insurance products
- Submit insurance applications
- Pay policy premium
- View active and expired policies
- Submit claims with supporting documents
- Track claim status

Agent

- Review insurance applications
- Approve or reject applications
- Manage assigned customers
- Track earned commissions

Claims Officer

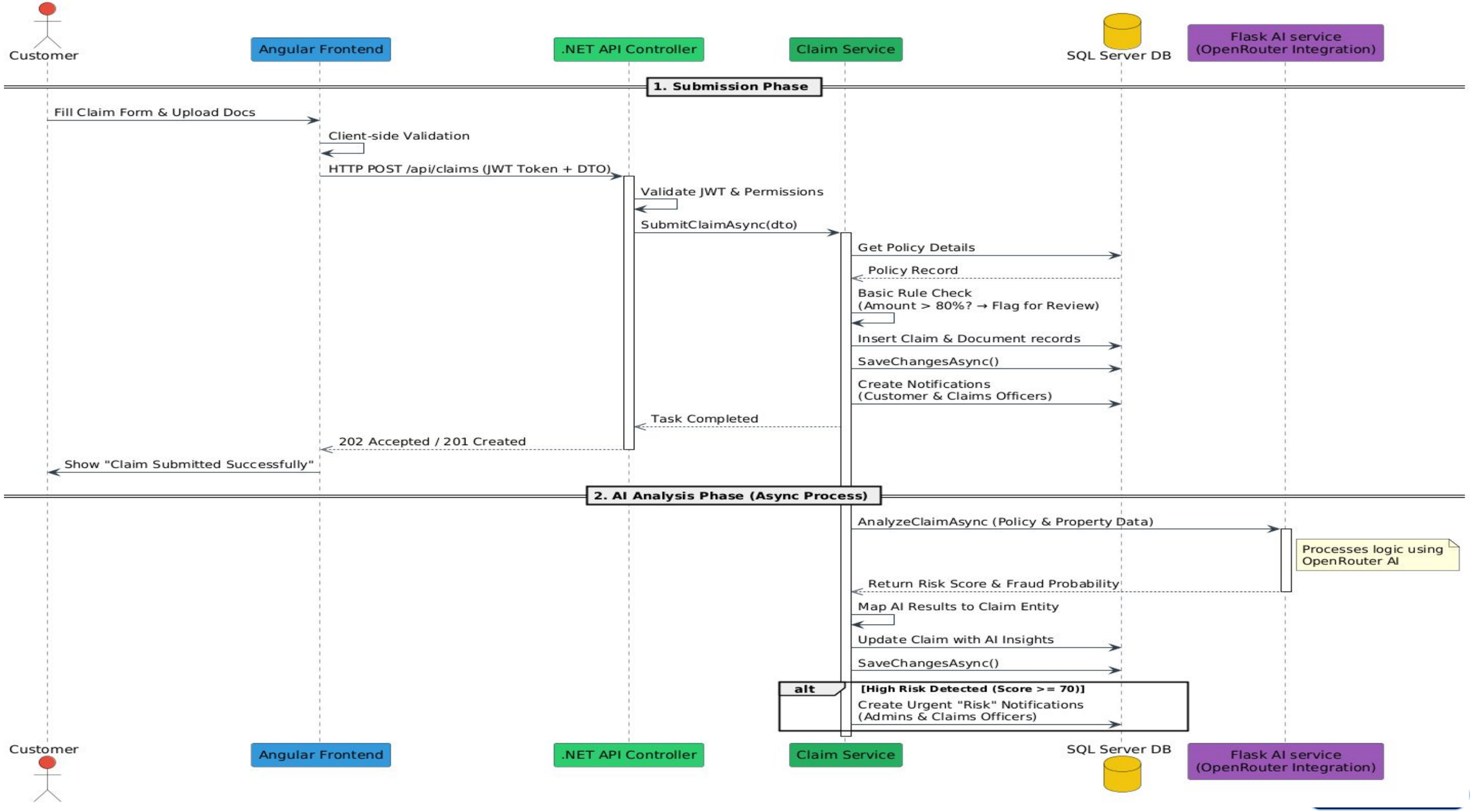
- Review submitted claims
- View AI risk score and fraud probability
- Prioritize claims based on risk
- Approve, reject, or partially settle claims

Admin

- Monitor system-wide operations
- View AI analytics dashboard
- Analyze fraud trends
- Evaluate agent performance
- Monitor high-risk claims
- Manage product configurations

Core Entities & Descriptions

Entity	Description	Key Attributes
User	Central identity table for all actors	Name, Email, Role, IsActive
Policy Application	Digital request form for new insurance	ProductId, CustomerId, MarketValue, Status
Policy	Active insurance contract	PolicyNumber, CoverageAmt, Premium, EndDate
Claim	Incident report filed by customer	ClaimAmount, AI Risk Score, AI Fraud Prob, Status
Agent	Professionals managing customers	LicenseNumber, JoinedDate, UserId
Property	Insured asset (Home/Business)	Address, MarketValue, RiskZone, YearBuilt
Payment	Financial history of premiums	Amount, PaymentDate, PaymentStatus
Commission	Agent earnings per policy	CommissionRate, CommissionAmount, IsPaid


Customer
Angular Frontend
.NET API Controller
Claim Service
SQL Server DB
Flask AI service
(OpenRouter Integration)
1. Submission Phase
Fill Claim Form & Upload Docs
Client-side Validation
HTTP POST /api/claims (JWT Token + DTO)
Validate JWT & Permissions
SubmitClaimAsync(dto)
Get Policy Details
Policy Record
Basic Rule Check
(Amount > 80%? → Flag for Review)
Insert Claim & Document records
SaveChangesAsync()
Create Notifications
(Customer & Claims Officers)
Task Completed
202 Accepted / 201 Created
Show "Claim Submitted Successfully"
2. AI Analysis Phase (Async Process)
AnalyzeClaimAsync (Policy & Property Data)
Processes logic using
OpenRouter AI
Return Risk Score & Fraud Probability
Map AI Results to Claim Entity
Update Claim with AI Insights
SaveChangesAsync()
alt
[High Risk Detected (Score >= 70)]
Create Urgent "Risk" Notifications
(Admins & Claims Officers)
Customer
Angular Frontend
.NET API Controller
Claim Service
SQL Server DB
Flask AI service
(OpenRouter Integration)

Testing Strategy

Unit testing (xUnit, Moq)

AI fallback testing

Frontend Testing(Jasmine,Karma)

Key Differentiators

- AI risk prioritization
- Explainable AI model
- Risk Analysis for Claims
- Hybrid Intelligent Fraud Detection
- Admin Business Intelligence

Future Enhancements

ML-based fraud model

Renewal automation

Policy endorsement feature

Document validation using AI

Customer support using a Bot

Thank You

We sincerely thank **The Hartford** for providing this valuable opportunity and supporting learning and innovation.

Special thanks to **GUVI and the entire team** for organizing and guiding this program.

We extend our gratitude to:

- **Surya Thammiraju** – Head & Managing Director, India, The Hartford
  for his leadership and encouragement.

- **Ranjan Bhatnagar** – Microsoft Certified Instructor
  for sharing valuable knowledge and insights.

- **Koti Rama Murthy Davuluri** – Senior Staff Software Engineer, The Hartford
  for continuous guidance and technical support.

🙏 **Thank you for your support and mentorship.**